

Embedded Control and ROS 2 Integration of a Custom 5-DOF Robot Arm

Student Project for EE/AU (5XEB0)

Aarrav Anil

*Department of Electrical Engineering
Eindhoven University of Technology
Student ID: 2075970*

Supervisors:

Dr. Dip Goswami (Resp.)
Aneesh Deogan (PhD Candidate)
Eindhoven University of Technology

Abstract—Reliable embedded control of robotic manipulators is dependent on real time closed loop algorithms within a decoupled system architecture. This paper details the electromechanical modeling, closed loop control design and software validation for a custom 5 Degree of Freedom (DOF) robot arm. By transitioning to a distributed software framework utilizing ROS 2 and microROS, high level motion planning on a Raspberry Pi is decoupled from the low level 200 Hz Proportional Derivative (PD) controllers running on a Teensy microcontroller. Decoupling is necessary to separate resource heavy, high level processing from the strict timing requirements of low level hardware control loops to prevent latency causing actuator instability. Furthermore, the integration of an embedded cubic polynomial trajectory generator along with an Exponential Moving Average (EMA) filter improved target tracking stability and quadrature encoder signal feedback. Empirical step response results validate an accurate digital twin of the joint actuators, providing a robust, modular platform for future kinematic expansions.

Index Terms—Embedded Control, ROS 2, micro-ROS, Digital Twin, Distributed Architecture, Trajectory Generation, PD Tuning

I. INTRODUCTION

Robotic manipulators require highly accurate and real time control over multiple actuator driven joints to interact safely and predictably with their environment. A major challenge in designing these systems is the resource conflict between high level tasks such as motion planning, inverse kinematics and user interfacing, and the strict timing constraints required by low level hardware actuator control loops [1].

Industry standard robotic systems rarely execute high frequency motor pulse width modulation (PWM) and closed loop control on the same processor handling high level tasks [2]. Instead, they utilize a decoupled structure to guarantee, real time execution. Furthermore, driving mechanical joints with raw step inputs leads to aggressive jerk, rapid structural degradation and significant transient overshoot [3]. Mitigating these challenges needs the use of continuous joint trajectory profiles that smooth out acceleration [4], along with precise system identification to validate the system's dynamic constraints with a digital twin [5].

The primary objective of this project is to utilize a custom designed 5 DOF robot arm shown in Figure 1 as an exper-

imental platform to develop, optimize and evaluate a fully distributed hardware and software ecosystem. Beyond evaluating the implementation of a real time communication layer, this work focuses on implementing a complete mechatronic framework: from sensor filtering and smooth cubic polynomial trajectory generation on an embedded microcontroller to parameter identification using a Simscape digital twin.

II. SYSTEM ARCHITECTURE AND INTEGRATION

To resolve the timing conflicts between high level processing and low level control, the project's software architecture was developed into a distributed architecture using ROS 2 (Robot Operating System) as illustrated in Figure 2.

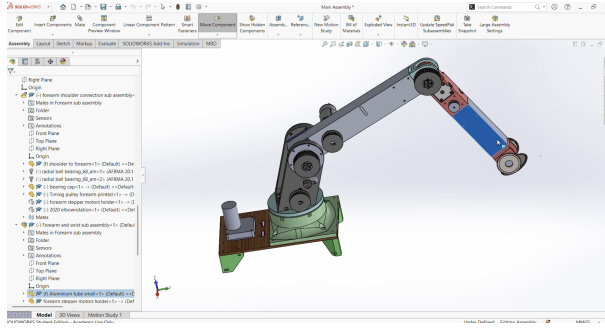
A. High-Level Processing Layer

The high level logic is handled by a Raspberry Pi 4 operating on Ubuntu Server 22.04 with ROS 2 Humble. Because Linux is a non real time operating system (RTOS), it is unsuitable for microsecond level hardware control. Instead, the Raspberry Pi serves purely as the operational brain. It hosts the ROS 2 Command Node, which publishes target position setpoints indicating the desired base target, shoulder target and trajectory duration to a designated ROS topic.

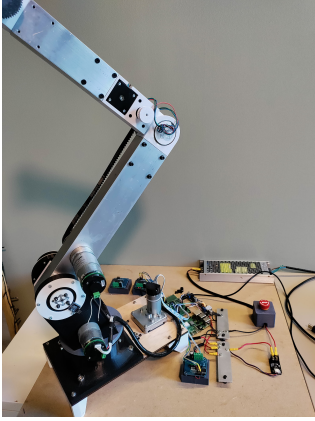
B. Embedded Microcontroller Layer

The physical execution is offloaded to a Teensy microcontroller running microROS. The Teensy operates a microROS subscriber node that actively listens to the target position topic. Once a command is read, the microcontroller calculates the necessary motion profile and executes a closed loop PD position controller at a strict loop rate of 200 Hz (a 5 ms control interval).

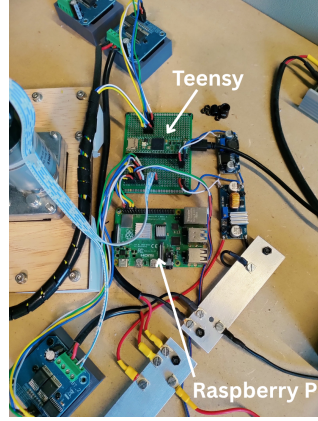
Simultaneously, a microROS publisher node gathers telemetry from the joint encoders. This continuous stream of data containing the actual joint position, the filtered velocity and the position error is published back to a Joint feedback topic on the Raspberry Pi. This data is then recorded via `rosbag2` allowing for data logging and performance analysis in MATLAB.



(a) SolidWorks CAD Model



(b) Physical 5-DOF Prototype



(c) Mechatronic Control Layer

Fig. 1: The custom 5 DOF robot arm hardware, showcasing the mechatronics design process from CAD to physical implementation.

III. ELECTROMECHANICAL MODELING

To develop an effective and safe controller before deploying it on fragile physical hardware, an accurate “digital twin” of the joint actuator was developed using Simulink and Simscape.

A. Digital Twin Parameters

The simulated model illustrated in Figure 3 was made to take a raw PWM signal as the input and output the motor’s angular velocity in degrees per second (Deg/s). The accuracy of this simulation was dependent on empirically tuning of five electromechanical parameters to perfectly mirror the real world robotic joint:

- **Armature Inductance:** Modifying this defined the electrical time constant, varying the micro delay in current buildup immediately after a PWM step.
- **Stall Torque:** Tuning this parameter calibrated the theoretical maximum turning force, shaping the slope of the simulated acceleration curve and rise time.
- **No Load Speed:** This limited the theoretical maximum steady state velocity of the motor in the absence of mechanical resistance.
- **Rotor Inertia:** Tuning this mechanical time constant simulated the physical mass and inertia of the 3D printed

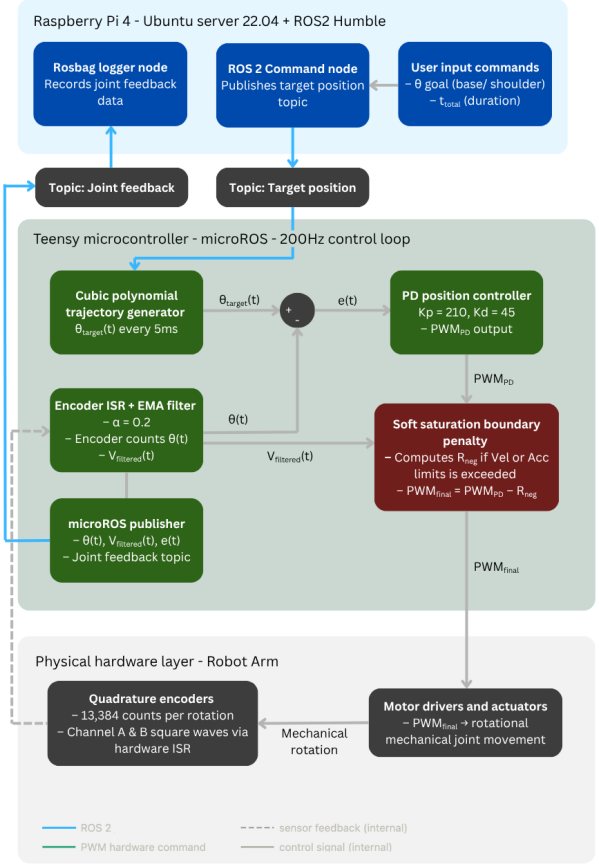


Fig. 2: System Block diagram: ROS 2 Architecture with the Raspberry Pi 4 and the Teensy microcontroller.

arm assemblies, changing how slowly the system ramps up and decelerates.

- **Gearbox (Coulomb) Friction:** This parameter was tuned to simulate parasitic frictional losses, lowering the maximum achievable steady state speed and defining a minimum PWM starting threshold to overcome static friction.

The identified parameters can be seen in Table I

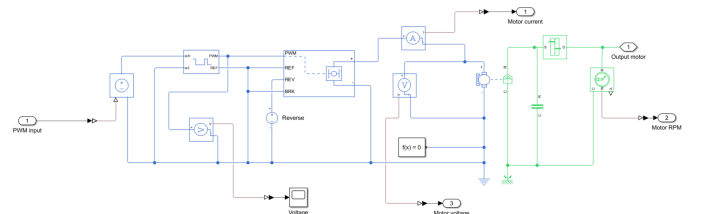


Fig. 3: Simscape diagram modeling the joint actuator

IV. CONTROL STRATEGY AND SIGNAL PROCESSING

A. Velocity Calculation and Signal Filtering

Accurate closed loop position and velocity control requires clean feedback. The custom 5 DOF arm uses high resolution quadrature encoders generating 13,384 pulses (counts) per

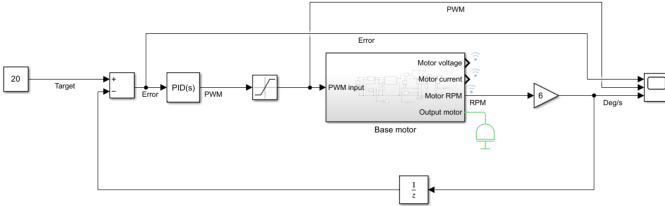


Fig. 4: Simulink block diagram mapping the closed loop PD controller alongside the Simscape actuator model.

TABLE I: Identified Actuator Parameter Values

Parameter	Symbol	Identified Value (Units)
Armature Inductance	L_a	1.1 mH
Stall Torque	τ_{stall}	0.22 Nm
No-Load Speed	ω_0	659 rad/s
Rotor Inertia	J_m	$1.43 \times 10^{-5} \text{ kg} \cdot \text{m}^2$
Coulomb Friction	T_f	$4.9 \times 10^{-3} \text{ Nm}$

rotation. These encoders are wired to digital pins on the Teensy microcontroller capable of hardware interrupts. When the encoder pulse is RISING or FALLING, an Interrupt Service Routine (ISR) is called to increment or decrement a count variable which keeps track of the current position of joints relative to the initial position. The encoder position updates by the ISR according to the phase shift ϕ between channel A and B of the quadrature encoder as shown in Figure 5:

$$\text{count} = \begin{cases} \text{count} + 1 & \text{if } \phi = +90^\circ \text{ (clockwise)} \\ \text{count} - 1 & \text{if } \phi = -90^\circ \text{ (counterclockwise)} \end{cases}$$

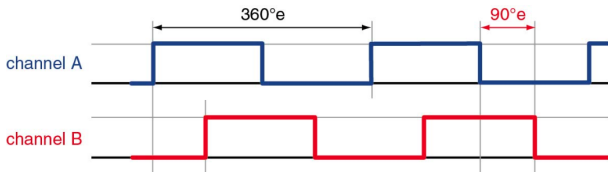


Fig. 5: Quadrature encoder output signal

Three discrete velocity estimation and signal processing techniques were implemented and evaluated on the microcontroller to derive a clean velocity feedback signal for the closed loop controller as plotted in Figure 6:

- 1) **Counts between time period (Method 1):** The number of encoder counts (ΔCounts) accumulated within a strict $\Delta t = 5 \text{ ms}$ time window was measured to calculate velocity via:

$$v_{\text{raw1}}(t) = \frac{\Delta\text{Counts}}{\text{CountsPerRot}} \times \frac{360^\circ}{\Delta t} \quad (1)$$

While this method introduced zero phase lag, it suffered from quantization noise issues. Because the count changes were discrete integers over a short window, the resulting raw velocity data was stepped and noisy,

which introduced actuator jitter if fed directly into the derivative term.

- 2) **Time between pulses (Method 2):** The time interval (Δt_{pulse}) between two consecutive encoder pulses was captured using high resolution hardware timers to calculate instantaneous velocity:

$$v_{\text{raw2}}(t) = \frac{1}{\text{CountsPerRot}} \times \frac{360^\circ}{\Delta t_{\text{pulse}}} \quad (2)$$

Although highly responsive at low velocities, this method was extremely sensitive to vibrations, high frequency structural noise and slight mechanical variations in the encoder disk spacing. At higher speeds, the tiny time intervals between pulses amplified tiny measurement variances into massive velocity spikes, introducing severe noise into the control loop.

- 3) **Counts between time period with Exponential Moving Average Filtering (Method 3):** To balance these trade-offs, Method 1 was coupled with a first order Exponential Moving Average (EMA) filter:

$$v_{\text{filtered}}(t) = \alpha \cdot v_{\text{raw1}}(t) + (1 - \alpha) \cdot v_{\text{filtered}}(t - \Delta t) \quad (3)$$

where α is the non dimensional smoothing factor.

Finally, Method 3 was selected for physical deployment. By performing step response testing, a coefficient of $\alpha = 0.2$ was chosen. This parameter was the optimal compromise as it functioned as a low pass filter that attenuated the high frequency quantization noise of the fixed window readings, while keeping the phase lag minimal enough to preserve the stability of the 200 Hz closed loop controller.

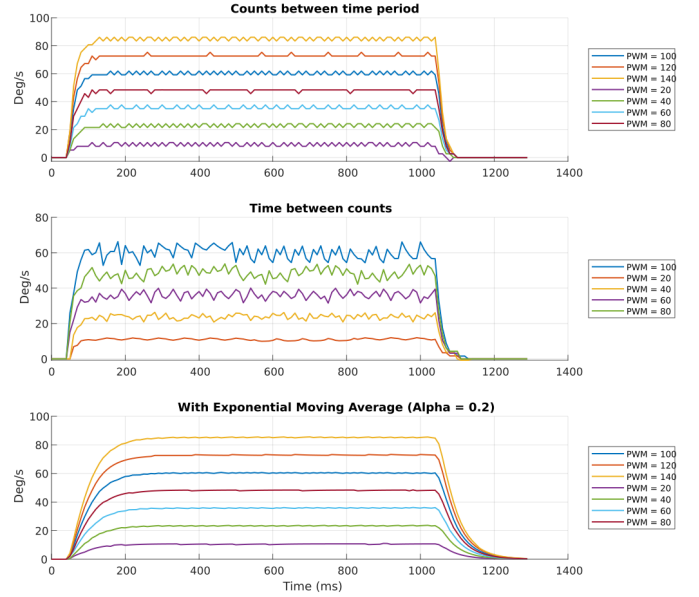


Fig. 6: Comparison of encoder signal processing: Raw calculations vs. Exponential Moving Average ($\alpha = 0.2$) filter output across varying PWM limits.

B. Cubic Polynomial Trajectory Generation

To prevent mechanical damage and extreme current spikes, the system moves from point A to point B using a continuous trajectory rather than attempting to track instantaneous step inputs.

A cubic polynomial trajectory generator was implemented in C++ on the Teensy. The generator assumes that both the initial and final mechanical accelerations are zero to ensure smooth departures and arrivals. The interpolated step t_{norm} is calculated based on the elapsed time relative to the desired duration:

$$t_{norm} = \frac{t_{elapsed}}{t_{total}} \quad (4)$$

The intermediate position target is then generated via the polynomial:

$$\theta_{target} = \theta_{init} + (\theta_{final} - \theta_{init}) \times (3t_{norm}^2 - 2t_{norm}^3) \quad (5)$$

C. PD Controller Tuning and Soft Saturated Boundary Constraints

With the trajectory continuously updating the set point every 5 ms, Proportional, Derivative (PD) position controllers ensure the motors accurately follow the path. The controllers were initially tuned within the Simulink digital twin to establish baseline parameters that prioritized a minimized settling time (t_s) and reduction of overshoot (M_p).

Upon deployment to the physical hardware, it was important to ensure the joints did not exceed safe operational bounds. To enforce this, a proportional penalty methodology was introduced to actively constrain the joint velocity and acceleration. Manually defined safety constants set the maximum allowable velocity (v_{max}) and acceleration (a_{max}) for each specific joint.

During the 200 Hz control loop, if the actual filtered velocity $v(t)$ or acceleration $a(t)$ exceeds these predefined limits, a proportional penalty (the negative reward, R_{neg}) is dynamically calculated:

$$R_{neg} = \lambda_v \max(0, |v(t)| - v_{max}) + \lambda_a \max(0, |a(t)| - a_{max}) \quad (6)$$

where λ_v and λ_a represent the proportional penalty weights for velocity and acceleration respectively. They function as dimensional scaling gains that map tracking errors to the actuator's PWM command space. This penalty is then applied to oppose the magnitude of the baseline PD control output (PWM_{PD}), effectively damping the final PWM signal (PWM_{final}) sent to the motor driver:

$$PWM_{final} = PWM_{PD} - R_{neg} \quad (7)$$

This methodology acts as an active software brake. It allows the PD controller to track the generated polynomial path under nominal conditions, but suppresses the PWM output if the physical joint attempts to violate its safe mechanical limits. The final hardware tuned parameters established for the dual joint movement were $K_p = 210$ and $K_d = 45$. The integral term (K_i) was intentionally omitted to prevent windup instability given the dynamic, constantly shifting nature of the trajectory set points.

V. RESULTS AND CRITICAL ANALYSIS

A. Digital Twin Validation and Analysis

To validate the digital twin, physical open loop step response tests were conducted and mapped against the Simulink output as illustrated in Figure 7. The Root Mean Square Error (RMSE) was utilized as the primary validation metric.

At low input ranges, the model proved exceptionally accurate. For example, at a PWM signal of 20, the RMSE between the physical robot and the simulation was low at 0.35 Deg/s. However, analysis at higher speeds showed a growing deviation. As the input reached a PWM of 120, the RMSE steadily increased to a peak of 2.25 Deg/s.

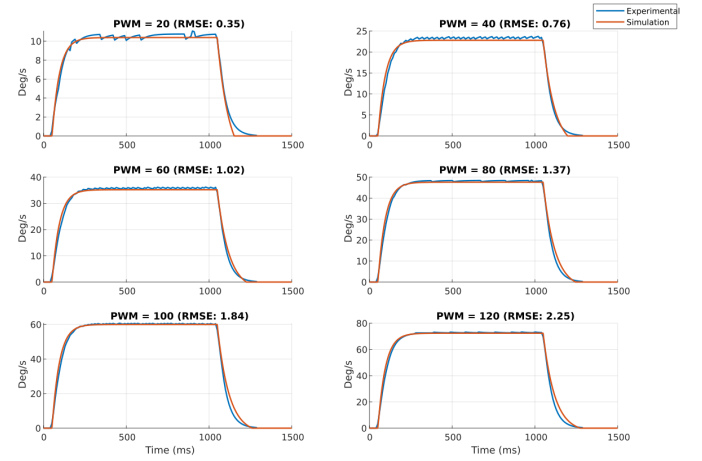


Fig. 7: Empirical validation of the Simulink digital twin against physical hardware across various PWM levels, displaying the associated RMSE.

This error deviation at higher velocities is due to purely linear modeling, not considering non linear dynamics inherent in the physical system. At higher rotational speeds, dynamic viscous damping (friction that scales disproportionately with velocity) and back EMF variations introduce physical resistance not perfectly captured by the base parameters. Also, the 5 DOF arm uses 3D printed structural components and timing belts. Higher velocities cause vibrations and belt elasticity, resulting in high frequency noise on the physical encoder data that simulation fails to consider. Despite this deviation at the upper boundaries, the digital twin provided a reliable baseline for deriving stable Kp and Kd parameters.

B. Trajectory Tracking Performance

Testing the integrated system and analyzing experimental data confirmed that under standard trajectories, the Teensy based PD controllers successfully tracked the cubic polynomial setpoints without lagging.

As illustrated in Figure 8, when tracking a step from 0° to -20° over a relaxed 500 ms profile, the actual joint position closely follows the target trajectory, without the transient overshoot observed with raw step inputs.

To test the upper operational boundaries of the physical joint, a high speed tracking experiment was conducted where

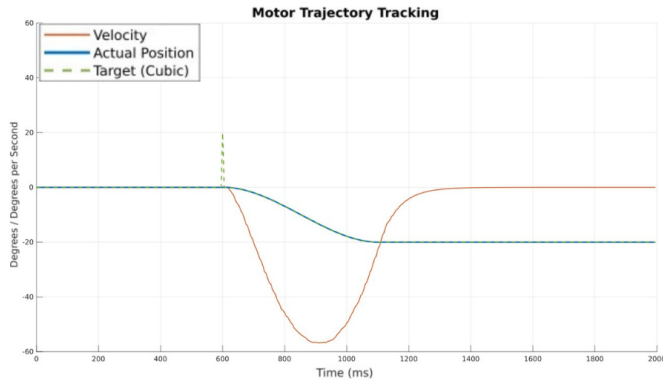


Fig. 8: Experimental motor trajectory tracking demonstrating the physical joint accurately tracking a nominal cubic polynomial reference profile (0° to -20° over a 500 ms window).

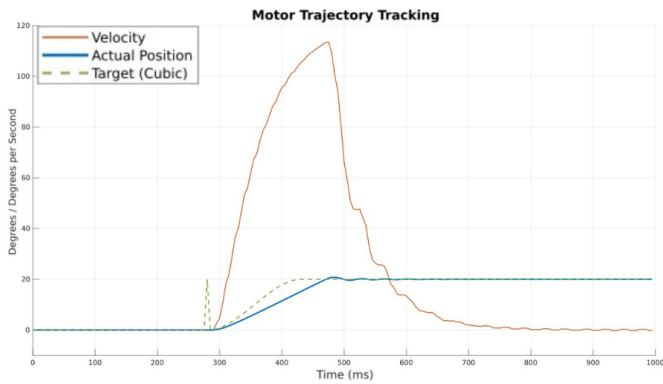


Fig. 9: Experimental motor trajectory tracking under high speed reference constraints (0° to 20° in 120 ms), illustrating the transient phase lag and velocity profile.

the target command forced a 0° to 20° displacement within a short 120 ms window, as shown in Figure 9. Under these severe timing constraints, a noticeable transient phase lag emerges between the target trajectory (dashed green) and the actual position (solid blue).

This tracking divergence is a direct consequence of the physical bandwidth limitations of the actuator assembly. Because the time window is so narrow, the controller commands a rapid control effort, driving the motor’s angular velocity to a peak of approximately $115^\circ/\text{s}$ (solid orange). At this operating point, the combination of back EMF voltage drop, physical rotor inertia and structural timing belt compliance prevents the motor from generating the instantaneous acceleration required to match the aggressive cubic curve.

Despite the visible phase lag during the transient acceleration phase, the controller shows desirable steady state behavior. The system dampens out smoothly without experiencing unstable oscillations or overshoot, reaching the target position after approximately 230 ms from start. This validates that the damping introduced by the $\alpha = 0.2$ EMA filter and the omitted integral term maintains system stability even under actuator

saturation.

VI. CONCLUSIONS AND FUTURE WORK

This project successfully completed the design, optimizations and deployment of a modular, closed loop distributed control system for an embedded robotics platform. The main contributions of this research are:

- **Effective Distributed Architecture:** Successfully realized a real time ROS 2 and microROS framework that cleanly separates low level Teensy hardware execution from Raspberry Pi trajectory planning and user interface.
- **Optimized Motion Profiles:** The mathematical integration of cubic polynomial trajectories eliminated mechanical shocks and minimized overshoot during joint transitions.
- **Encoder Signal Processing:** Implementing an $\alpha = 0.2$ EMA filter suppressed quadrature encoder high frequency noise without introducing severe control phase delay.
- **Accurate Digital Twin:** The Simulink model was empirically validated, proving that the parameter identification strategy accurately represents the physical robot.

Future work will focus on expanding this scalable ROS 2 and microROS architecture to control all 5 axes simultaneously. Building upon this joint level control, forward and inverse kinematic algorithms will be added to allow the manipulator’s end effector to track continuous Cartesian paths in physical space.

REFERENCES

- [1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2005.
- [2] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, pp. 1–11, May 2022.
- [3] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [4] A. Piazzzi and A. Visioli, “Global minimum-jerk trajectory planning of robot manipulators,” *IEEE Transactions on Industrial Electronics*, vol. 47, no. 1, pp. 140–149, Feb. 2000.
- [5] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Hoboken, NJ: John Wiley & Sons, 2006.